TRABAJO PRÁCTICO · UNIDAD 7

NoSQL y Nuevas Formas de Almacenamiento | Food Store

> **Asignatura:** Bases de Datos II — Tecnicatura Universitaria en Programación
> **Equipo de Trabajo:** Máximo Franco, Andrés Novello y Octavio Fiore
> **Entorno Tecnológico:** PostgreSQL 17+ (origen relacional) | MongoDB 7+ (destino documental) | Node.js / mongosh
> **Repositorio Oficial:** https://github.com/MaxiFranco05/DB2 (rama `main`)
> **Entregables de Código:** `consultas_mongodb.js`, `consultas_postgresql.sql`, `docs/DUIA/DUIA-TP-Unidad7.md`

1. Introducción y Alcance del Trabajo

El presente trabajo práctico implementa la migración, modelado documental y evaluación comparativa entre **PostgreSQL** y **MongoDB** para el subconjunto operacional de **pedidos y detalle de pedidos** de la plataforma *Food Store*.

Se seleccionó una muestra real de **5 pedidos** (IDs `756504`, `756505`, `756506`, `756507` y `756508`) concentrados en el cliente `80584`, abarcando los estados `TERMINADO`, `CONFIRMADO` y `PENDIENTE`, y diversas formas de pago (`EFECTIVO`, `TARJETA`, `TRANSFERENCIA`).

2. Parte 1 · Modelo Documental y Migración (Consignas 1, 2 y 3)

2.1. Mapeo Relacional a Documental (Consigna 1)

En PostgreSQL, recuperar un pedido completo exige la unión de 4 tablas: `pedido`, `detalle_pedido`, `usuario` y `producto`. Para MongoDB, se diseñó la colección `pedidos` bajo un esquema de **embedding completo** en 3 niveles:

- **Campos raíz:** `pedido_id_origen` (PK relacional conservada para trazabilidad), `fecha` (ISODate), `estado`, `forma_pago` y `total`.
- **Objeto embebido `cliente`:** `usuario_id_origen`, `nombre_completo` y `mail`. *Por directiva estricta de seguridad y privacidad, se excluyó la columna `contrasena`.*
- **Arreglo de subdocumentos `items`:** Cada posición almacena `producto_id_origen`, `nombre`, `cantidad`, `precio_unitario_historico` y `subtotal`.

Estructura BSON Canónica:

```
{
  "_id": ObjectId("..."),
  "pedido_id_origen": 756505,
  "fecha": ISODate("2026-06-08T03:00:00Z"),
  "estado": "CONFIRMADO",
  "forma_pago": "TARJETA",
  "total": 16307.74,
  "cliente": {
    "usuario_id_origen": 80584,
    "nombre_completo": "Usuario584 Apellido584",
    "mail": "usuario584@test.com"
  },
  "items": [
    { "producto_id_origen": 243882, "nombre": "Producto 43882", "cantidad": 2, "precio_unitario_historico"
```

```
    { "producto_id_origen": 236648, "nombre": "Producto 36648", "cantidad": 3, "precio_unitario_historico"
    { "producto_id_origen": 220320, "nombre": "Producto 20320", "cantidad": 3, "precio_unitario_historico"
    { "producto_id_origen": 221809, "nombre": "Producto 21809", "cantidad": 4, "precio_unitario_historico"
  ]
}
```

2.2. Migración Efectuada (Consigna 2)

La base `foodstore_nosql` y la colección `pedidos` fueron creadas y pobladas mediante el script `consultas_mongodb.js` utilizando `insertMany()` de forma idempotente con los 5 pedidos de muestra, garantizando precios históricos congelados.

2.3. Reglas de Integridad y Justificación de Diseño (Consigna 3)

Reglas de integridad preservadas:

1. **RI-M1 (Congelamiento de precios y consistencia matemática):** En PostgreSQL el trigger `trg_03_calcular_subtotal` congela el precio al insertar. En MongoDB, la migración aseguró que `precio_unitario_historico` no se recalcule con precios actuales de producto, preservando la igualdad exacta: `subtotal = cantidad * precio_unitario_historico` y `total = sum(subtotal)`.
2. **RI-M2 (Atomicidad y no vaciedad de líneas):** Se garantizó que ningún pedido se persista sin ítems (`len(items) >= 1`), asegurando la atomicidad de la orden comercial en una sola operación documental.

Justificación del Embedding vs. Referencing: Se eligió embedding para optimizar la *localidad de datos* ("datos que se leen juntos se almacenan juntos"), permitiendo recuperar el ticket completo en una única lectura $O(1)$ sin JOINs ni `$lookup`, y asegurando la *inmutabilidad histórica* de la venta ante modificaciones futuras en el catálogo maestro.

3. Parte 2 · Consultas y Verificación (Consignas 4 y 5)

3.1. Consultas Ejecutadas en MongoDB (Consigna 4)

Las 4 consultas fueron implementadas en `consultas_mongodb.js` y ejecutadas con proyecciones explícitas:

- **Q-M1: Listar todos los pedidos (identificador y estado):**
  `db.pedidos.find({}, { pedido_id_origen: 1, estado: 1, _id: 0 }).sort({ pedido_id_origen: 1 });`
  Resultado: Retorna los 5 documentos de la muestra.
- **Q-M2: Filtrar pedidos por estado 'CONFIRMADO':**
  `db.pedidos.find({ estado: "CONFIRMADO" }, { pedido_id_origen: 1, estado: 1, fecha: 1, total: 1, _id: 0 });`
  Resultado: Retorna 2 pedidos: IDs `[756505, 756508]`.
- **Q-M3: Buscar pedidos con producto determinado (243882):**
  `db.pedidos.find({ "items.producto_id_origen": 243882 }, { pedido_id_origen: 1, estado: 1, "items.$": 1, _id: 0 });`
  Resultado: Retorna 4 pedidos: IDs `[756505, 756506, 756507, 756508]` con proyección posicional del ítem.
- **Q-M4: Filtrar pedidos con total superior a $15.000:**
  `db.pedidos.find({ total: { $gt: 15000 } }, { pedido_id_origen: 1, estado: 1, total: 1, _id: 0 }).sort({ total: -1 });`
  Justificación del umbral: Distribuye la muestra entre pedidos chicos (<$15k: 756504 por $4.517,28 y 756508 por $13.052,26) y medianos/altos (>$15k: 756507 por $15.128,05, 756505 por $16.307,74 y 756506 por $18.092,69).
  Resultado: Retorna 3 pedidos (756506, 756505, 756507).

3.2. Consultas Equivalentes en PostgreSQL y Tabla de Verificación (Consigna 5)

En PostgreSQL se ejecutaron las consultas relacionales equivalentes en `consultas_postgresql.sql` (Q-PG1 para estado `CONFIRMADO` agrupando por pedido con `SUM(dp.subtotal)`, y Q-PG2 para producto `243882` con `SELECT DISTINCT`):

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?	Identificadores Recuperados
Estado seleccionado (`CONFIRMADO`)	**2**	**2**	**Sí** ✅	`[756505, 756508]`
Producto seleccionado (`243882`)	**4**	**4**	**Sí** ✅	`[756505, 756506, 756507, 756508]`

> **Resultado del Contraste Empírico:** Se verificó una **coincidencia exacta del 100%** entre ambos motores (0 filas de diferencia con `EXCEPT` bidireccional). El precio unitario histórico para el producto `243882` en PostgreSQL ($3628.43) resultó idéntico al valor conservado en MongoDB.

4. Parte 3 · Evaluación de la Tecnología e IA (Consignas 6 y 7)

4.1. Prompt Formulado para la Comparativa de IA (Consigna 6)

> *"Compara técnicamente PostgreSQL y MongoDB para el almacenamiento y consulta de pedidos en una aplicación gastronómica (Food Store). Considera que el modelo relacional actual utiliza 4 tablas (pedido, detalle_pedido, usuario, producto), que el acceso más frecuente es la recuperación de pedidos completos para despacho y auditoría, y que es imperativo conservar la coherencia transaccional de cantidades y precios históricos."*

4.2. Contraste de Afirmaciones Técnicas de la IA (Consigna 7)

- **Afirmación 1 de la IA:** "*MongoDB no soporta transacciones ACID y no garantiza consistencia en operaciones multi-documento.*"
  Contraste con Documentación Oficial: Falsa / Desactualizada. La documentación oficial de MongoDB documenta soporte completo para transacciones ACID multi-documento desde la versión 4.0/4.2 mediante `session.startTransaction()`. No obstante, el laboratorio demostró que con el modelo embebido, la creación de un pedido y sus líneas se resuelve de forma atómica en un único documento, prescindiendo del overhead de transacciones multi-documento.
- **Afirmación 2 de la IA:** "*El embedding duplica datos innecesariamente y desperdicia almacenamiento frente a tablas normalizadas.*"
  Contraste con Documentación Oficial: Conceptualmente errónea. La guía oficial de diseño de datos de MongoDB prescribe el embedding para relaciones uno a pocos fuertemente cohesionadas. Confunde redundancia con *inmutabilidad histórica*: congelar el precio unitario y el nombre del producto en el pedido es un requisito de negocio para que cambios futuros en catálogo no corrompan pedidos ya facturados.

4.3. Conclusión Técnica Final (Consigna 7 — Límite: 200 palabras)

La evaluación comparativa entre PostgreSQL y MongoDB para el subconjunto de pedidos de Food Store demostró que la elección del motor debe fundamentarse en el patrón de acceso y los requisitos de consistencia, descartando dogmas tecnológicos.

Para la gestión operativa de pedidos completos, **MongoDB con modelo embebido presenta ventajas técnicas determinantes**: al almacenar cliente, líneas y totales en un único documento, elimina los costosos `JOIN` relacionales, resolviendo las consultas de despacho y lectura de tickets con lecturas atómicas de disco en complejidad $O(1)$.

Sin embargo, para la gestión del catálogo maestro, inventario concorrente de stock y la contabilidad global, **PostgreSQL resulta insustituible** gracias a su motor relacional ACID estricto, triggers automáticos y constraints declarativos que impiden sobreventas o anomalías lógicas.

Por lo tanto, la solución arquitectónica óptima para Food Store es un esquema de **persistencia políglota**: mantener a PostgreSQL como la fuente de la verdad (*Source of Truth*) transaccional para usuarios, productos y stock, y replicar o sincronizar asincrónicamente los pedidos finalizados hacia MongoDB para alimentar de forma ultra-rápida y escalable los paneles de despacho, consultas de clientes e histórico de compras.

5. Anexo de Entregables Generados

- `consultas_mongodb.js`: Carga inicial de muestra con `insertMany()`, consultas Q-M1 a Q-M4 y aserciones RI-M1 y RI-M2.
- `consultas_postgresql.sql`: Consultas relacionales equivalentes Q-PG1 y Q-PG2 con `\echo` y sin `SELECT *`.

- `docs/DUIA/DUIA-TP-Unidad7.md`: Declaración estricta de uso de IA en formato tabular vertical de dos columnas.
- `BITACORA.md`: Registro cronológico de auditoría y lectura crítica del equipo.